

```
137         this.teachingHours = teachingHours;
138         this.incentivePerHour = incentivePerHour;
139     }
140
141     public double calculateSalary() {
142         return getBasicSalary() + (teachingHours * incentivePerHour);
143     }
144 }
145
146 class TechnicalStaff extends Employee {
147
148     int overtimeHours;
149     double overtimeRate;
150
151     TechnicalStaff(int id, String name, String dept, double basicSalary,
152                   int overtimeHours, double overtimeRate) {
153
154         super(id, name, dept, basicSalary);
155         this.overtimeHours = overtimeHours;
156         this.overtimeRate = overtimeRate;
157     }
158
159     public double calculateSalary() {
160         return getBasicSalary() + (overtimeHours * overtimeRate);
161     }
162 }
163
164 class VisitingFaculty extends Employee {
165
166     int lecturesHandled;
167     double paymentPerLecture;
168
169     VisitingFaculty(int id, String name, String dept, double basicSalary,
170                    int lecturesHandled, double paymentPerLecture) {
171
172         super(id, name, dept, basicSalary);
173         this.lecturesHandled = lecturesHandled;
174         this.paymentPerLecture = paymentPerLecture;
175     }
176
177     public double calculateSalary() {
178         return getBasicSalary() + (lecturesHandled * paymentPerLecture);
179     }
180 }
```

```
137         this.teachingHours = teachingHours;
138         this.incentivePerHour = incentivePerHour;
139     }
140
141     public double calculateSalary() {
142         return getBasicSalary() + (teachingHours * incentivePerHour);
143     }
144 }
145
146 class TechnicalStaff extends Employee {
147
148     int overtimeHours;
149     double overtimeRate;
150
151     TechnicalStaff(int id, String name, String dept, double basicSalary,
152                   int overtimeHours, double overtimeRate) {
153
154         super(id, name, dept, basicSalary);
155         this.overtimeHours = overtimeHours;
156         this.overtimeRate = overtimeRate;
157     }
158
159     public double calculateSalary() {
160         return getBasicSalary() + (overtimeHours * overtimeRate);
161     }
162 }
163
164 class VisitingFaculty extends Employee {
165
166     int lecturesHandled;
167     double paymentPerLecture;
168
169     VisitingFaculty(int id, String name, String dept, double basicSalary,
170                    int lecturesHandled, double paymentPerLecture) {
171
172         super(id, name, dept, basicSalary);
173         this.lecturesHandled = lecturesHandled;
174         this.paymentPerLecture = paymentPerLecture;
175     }
176
177     public double calculateSalary() {
178         return getBasicSalary() + (lecturesHandled * paymentPerLecture);
179     }
180 }
```

```
35     String searchId = "F101";
36     searchPatient(searchId);
37
38     displayAllPatients();
39
40     countDepartmentWisePatients();
41 }
42
43 static void registerPatient(String patientId, String patientName,
44
45     if (patientList.size() >= 100) {
46         System.out.println("Cannot register more than 100 patients");
47         return;
48     }
49
50     patientId = trimAll(patientId);
51     patientName = trimAll(patientName);
52     mobileNumber = trimAll(mobileNumber);
53     department = trimAll(department);
54     doctorName = trimAll(doctorName);
55
56     if (patientMap.containsKey(patientId)) {
57         System.out.println("Duplicate Patient ID. Registration failed");
58         return;
59     }
60
61     if (!isAlphabeticWithSpaces(patientName)) {
62         System.out.println("Invalid Patient Name. Registration failed");
63         return;
64     }
65
66     if (!mobileNumber.matches("\\d{10}")) {
67         System.out.println("Invalid Mobile Number. Registration failed");
68         return;
69     }
70 }
```

```
70
71     String deptUpper = department.toUpperCase();
72     String properName = toProperCase(patientName);
73
74     Patient patient = new Patient(patientId, properName, mobileNum
75     patientList.add(patient);
76     patientMap.put(patientId, patient);
77
78     System.out.println("Patient Registered Successfully.");
79 }
80
81 static void searchPatient(String patientId) {
82     patientId = trimAll(patientId);
83     Patient p = patientMap.get(patientId);
84
85     if (p == null) {
86         System.out.println("Patient Not Found");
87     } else {
88         System.out.println("Patient Details");
89         System.out.println("ID : " + p.patientId);
90         System.out.println("Name : " + p.patientName);
91         System.out.println("Mobile : " + p.mobileNumber);
92         System.out.println("Department : " + p.department);
93         System.out.println("Doctor : " + p.doctorName);
94         System.out.println("Name Length : " + p.patientName.replac
95     }
96 }
97
98 static void displayAllPatients() {
99     System.out.println("-----");
100    System.out.println("ID Name Mobile Department Doctor");
101    System.out.println("-----");
102    for (Patient p : patientList) {
103        System.out.println(p.patientId + " " + p.patientName + " "
104    }
```

```
117     System.out.println("Department Count");
118     for (Map.Entry<String, Integer> e : departmentCount.entrySet())
119         System.out.println();
120         System.out.println(e.getKey() + " : " + e.getValue());
121     }
122 }
123
124 static String trimAll(String s) {
125     return s == null ? "" : s.trim().replaceAll("\\s+", " ");
126 }
127
128 static boolean isAlphabeticWithSpaces(String name) {
129     return name.matches("[A-Za-z]+([A-Za-z]+)*");
130 }
131
132 static String toProperCase(String s) {
133     s = trimAll(s);
134     String[] parts = s.split(" ");
135     StringBuilder sb = new StringBuilder();
136     for (int i = 0; i < parts.length; i++) {
137         String w = parts[i].toLowerCase();
138         if (w.length() > 0) {
139             sb.append(Character.toUpperCase(w.charAt(0))).append(w
140                 .substring(1));
141             if (i != parts.length - 1) sb.append(" ");
142         }
143     }
144     return sb.toString();
145 }
```

your Code

```
1 import java.util.*;
2
3 class Book {
4     String bookid, title, author, category, publisher;
5
6     Book(String bookid, String title, String author, String category,
7         this.bookid = bookid;
8         this.title = title;
9         this.author = author;
10        this.category = category;
11        this.publisher = publisher;
12    }
13 }
14
15 public class Main {
16     public static void main(String[] args) {
17
18         Scanner sc = new Scanner(System.in);
19
20         ArrayList<Book> books = new ArrayList<>();
21         HashMap<String, Book> bookMap = new HashMap<>();
22         HashSet<String> authors = new HashSet<>();
23
24         while (true) {
25             System.out.println("\n1. Add Book");
26             System.out.println("2. Search Book by ID");
27             System.out.println("3. Display All Books");
28             System.out.println("4. Display Unique Authors");
29             System.out.println("5. Category-wise Book Count");
30             System.out.println("6. Exit");
31             System.out.print("Enter Choice: ");
32
33             int choice = sc.nextInt();
34             sc.nextLine();
```

input

stdin input

Output

```
Compilation Error:
C:\Windows\TEMP\sandbox_192472274
RA_6a6e2e30921805.95016410\Book.j
ava:15: error: class Main is
public, should be declared in a
file named Main.java
```

Visible Test Cases

- Test Case 1
- Test Case 2
- Test Case 3

0.00 / 20 marks

```
35  
36     switch (choice) {  
37  
38         case 1:  
39             System.out.print("Enter Book ID: ");  
40             String id = sc.nextLine();  
41  
42             if (bookMap.containsKey(id)) {  
43                 System.out.println("Duplicate Book ID!");  
44                 break;  
45             }  
46  
47             System.out.print("Enter Book Title: ");  
48             String title = sc.nextLine();  
49  
50             System.out.print("Enter Author Name: ");  
51             String author = sc.nextLine();  
52  
53             System.out.print("Enter Category: ");  
54             String category = sc.nextLine();  
55  
56             System.out.print("Enter Publisher: ");  
57             String publisher = sc.nextLine();  
58  
59             Book b = new Book(id, title, author, category, publisher);  
60  
61             books.add(b);  
62             bookMap.put(id, b);  
63             authors.add(author);  
64  
65             System.out.println("Book Added Successfully.");  
66             break;  
67  
68         case 2:  
69             System.out.print("Enter Book ID: ");
```

```
69         System.out.println("Enter Book ID: ");
70         String search = sc.nextLine();
71
72         if (bookMap.containsKey(search)) {
73             Book bk = bookMap.get(search);
74             System.out.println("Book ID : " + bk.bookId);
75             System.out.println("Title : " + bk.title);
76             System.out.println("Author : " + bk.author);
77             System.out.println("Category : " + bk.category);
78             System.out.println("Publisher : " + bk.publisher);
79         } else {
80             System.out.println("Book Not Found.");
81         }
82         break;
83
84     case 3:
85         for (Book bk : books) {
86             System.out.println(bk.bookId + " " + bk.title);
87         }
88         break;
89
90     case 4:
91         System.out.println("Unique Authors:");
92         for (String a : authors) {
93             System.out.println(a);
94         }
95         break;
96
97     case 5:
98         HashMap<String, Integer> categoryCount = new HashMap<>();
99
100         for (Book bk : books) {
101             categoryCount.put(bk.category,
102                 categoryCount.getOrDefault(bk.category, 0) + 1);
103         }
104     }
```



```
92         for (String a : authors) {
93             System.out.println(a);
94         }
95         break;
96
97     case 5:
98         HashMap<String, Integer> categoryCount = new HashM
99
100         for (Book bk : books) {
101             categoryCount.put(bk.category,
102                 categoryCount.getDefault(bk.category)
103             )
104
105             for (String c : categoryCount.keySet()) {
106                 System.out.println(c + " : " + categoryCount.g
107             }
108             break;
109
110     case 6:
111         System.out.println("Thank You");
112         sc.close();
113         return;
114
115     default:
116         System.out.println("Invalid Choice");
117     }
118 }
119 }
120 }
```